Lec – 30 Modularity in Modelling

1. Why Kernel Methods Are Not Modular

Kernel methods (like SVM with kernels RBF) are mathematically powerful
But they have some **limitations**:

(a) Not Modular

- You cannot build a complex model by combining simpler models.
- There is no concept of ***levels*** **of representation.**
- Everything is "flat" — one big transformation via the kernel.

(b) Model Bias Is Hidden - encoded inside the **kernel function**.

- But this bias is **not easily interpretable.**
- Example: Matérn kernels were specifically designed to impose "smoothness".
  But **in general**, if you have a desired bias, there is no easy method to directly encode it in a kernel.

(c) Technical Conditions

- Kernels must satisfy mathematical conditions like **Aronszajn's conditions** (positive definiteness).
- Designing good kernels is hard and was a major research effort (1995–2005) in vision, text, speech, biology.

---

2. Why Neural Networks ARE Modular

Neural networks are **modular by design**:

- Big models consist of simple building blocks (layers).
- You can compose operations again to build complex pattern recognizers.

This modularity is inspired by **how humans recognize patterns.**

---

3. Example: Recognizing Digits vs. Alphabets

Kernel/Linear Model Behaviour

Let's say a kernel/linear classifier is:

$$\hat{f}(x) = \arg \max_{y \in \{0,1,\ldots,9\}} w_y^\top \phi(x)$$

For this to work:

- $w_y^\top \phi(x)$ must be largest for the correct digit.
- This means: **the weight vector w_y must look like the digit "y"** in feature space.
- So, the model learns **shapes of digits**.

Now if we switch to alphabet recognition:

- The model again learns **shapes of alphabets**.
- **But nothing is reused** from digit learning because there is no hierarchy of shared patterns.

---

4. Motivation for Hierarchical Pattern Recognition

Real-world tasks often share **basic patterns**.

Example:

- Digits and alphabets both contain:
    - strokes,
    - curves,
    - horizontal lines,
    - edges, etc.

A model that learns *basic patterns first* and then builds complex patterns on top can **reuse knowledge**.

This motivates **modular / hierarchical models**, i.e., neural networks.

---

5. Formal Description of a Hierarchy of Patterns

Consider **level 1 patterns** represented by vectors:

$$w_i^{\{1\}}, \quad i = 1, \ldots, n_1$$

These are learned patterns over pixels.

Recognizing Level-1 Patterns

A pattern is detected in image x if:

$$(w_j^{\{1\}})^\top x > 0$$

But "> 0" is discontinuous, so we use a smooth-ish alternative:

$$\max(0,\ (w_j^{\{1\}})^\top x)$$

This is like ReLU.

If we stack all level-1 pattern vectors as columns:

$$W^{\{1\}} = [w_1^{\{1\}} \quad w_2^{\{1\}} \quad \cdots \quad w_{n_1}^{\{1\}}]$$

Then level-1 activations are:

$$x^{\{2\}} = \max(0,\ (W^{\{1\}})^\top x)$$

ReLU – Rectified Linear Unit

Level-2 Patterns

Now, take level-2 pattern vectors in matrix (W^{{2}}):

$$x^{\{3\}} = \max(0,\ (W^{\{2\}})^\top x^{\{2\}})$$

And so on…

Each level extracts more abstract patterns.

Eventually:

- **Last level patterns correspond to actual classes** (digits, alphabets).

But unlike kernel methods:

- **Intermediate patterns are meaningful**.
- They correspond to edges, curves, simple strokes, etc.

---

6. Why Modularity Is So Powerful

Because neural networks are modular:

- You can combine small models to build big systems.

Example:

1. **Encoder models**
   Convert raw data (text, speech, images) into meaningful embeddings.
2. **Decoder models**
   Convert embeddings back into text, speech, images.
3. **Encoder–Decoder systems**
   - Text → Image
   - Image → Text
   - Text → Speech
   - Speech → Text
     and so on.

This modularity is a **major reason modern AI works so well**.

Lec – 31 Deep Learning Models: Neural Networks

⭐ Deep Learning Models: Neural Networks

1. What is a Neural Network Model?

A neural network model is defined by:

✔ A DAG (Directed Acyclic Graph)

- Each node - function with parameters.
- Each directed edge (u -> v) means:
  - The output of node (u) is used as an input to node (v).

✔ Special Nodes

- Input nodes (input layer):
  - These nodes store the input (x).
- Output nodes (output layer):
  - These produce the final output (activations).

To define a neural network model, two things must be fixed:

- The DAG structure
- The parametrized functions at each node

Once these are fixed, different parameter values give different functions from the same model.

---

2. Example DAG

Given DAG: $x \to l_U \to a \to l_V \to a \to l_W$

Where:

- **Linear unit (linear layer):** $x \to l_U \to a \to l_V \to a \to l_W$
- **ReLU activation:**
  $$a(x) = \max(0, x)$$
  (applied elementwise)

This is exactly the example from previous lecture (hierarchical pattern recognition).

Function induced by the whole DAG

$$l_W\left(a\left(l_V\left(a\left(l_U(x)\right)\right)\right)\right)$$

This means:

- First apply linear transformation by (U)
- Apply ReLU
- Apply linear transformation by (V)
- Apply ReLU
- Final linear transformation by (W)

Interpretation

- Columns of (U): pattern detectors of level 1 (edges, curves).
- Columns of (V): pattern over patterns (more complex shapes).
- Columns of (W): patterns for final classes (digits, alphabets, etc.)

This DAG is a linear chain, and uses only:

- Linear layers
- ReLU activation

---

3. Multilayer Perceptron's (MLPs)

The above structure—repeated composition of:

$$a(l(x))$$

done (d) times, followed by output layer—is called:

✔ **Multilayer Perceptron (MLP)**

✔ **Feed-forward Neural Network (FFNN)**

Important Terminology

- **Layer:** one block of linear → activation
- **Layer activation:** the output of the layer
- **Depth:** number of intermediate layers (not counting output layer).
  In our example → (d)
- Width: the maximum number of neurons (columns) in any intermediate layer

---

- Early NN models (1990s) were shallow (1–2 layers).
- Reason: training deep networks was hard because optimization algorithms were weak.
- After 2010:
  1. Better SGD variants
  2. GPUs
  3. Large datasets
     → Deep networks became trainable and successful

---

6. Universal Approximation and Width vs Depth

✔ **Classical result (Hornik, 1989)**

A very wide, shallow network (depth = 1) can approximate any function.

This is like kernel methods:

- They also use extremely large (even infinite) feature spaces (flat structure).

✔ **Connections between wide networks and kernels**

Works like:

- **Neural Tangent Kernel (NTK)**
- Lazy training
  show mathematical links between very wide NNs and kernel methods.

✔ **More modern results: Depth also gives approximation power**

Width-limited deep networks are **also universal approximators** if width is above a minimum threshold.

Some papers even show:

Depth is **exponentially more powerful than width**
(e.g., Eldan & Shamir 2016)

Meaning:

- Adding layers can achieve functions that a shallow network would need exponential width to represent.

---

7. Estimation Error (Generalization)

Neural networks have similar estimation error behaviour as earlier models.

✔ **Example Result**

From Anthony & Bartlett (Theorem 21.5): **Estimation Error=R(f^)−R(f*) f* best possible f^ Learned**

$$O\left(d\sqrt{\frac{n}{m}}\right)$$

Where:

- (d) = depth + (n) = number of parameters + (m) = sample size

More depth increases error unless parameters are well regularized.

✔ **Norm-based bound (from Bach, Exercise 9.2)**

$$O\left(\frac{W^d R}{\sqrt{m}}\right)$$

Where:

- **(W)** = norm bound of parameters of each linear layer
- **(R)** = input radius

Depth increases the multiplicative factor (W^d), so depth can hurt estimation error.

Conclusion

There is a **bias–variance tradeoff**:

- More depth → more modeling power
- But also → more chance of overfitting unless regularized

---

8. Optimization Error

This is where neural networks behave very differently from convex models.

- Risk minimization is highly non-convex – (Many local minima)
- The classical table comparing GD/SGD convergence no longer holds

✔ Theoretical Results

- Even finding the global minimum of a neural network can be NP-hard (theorem 20.7 from Shalev-Shwartz & Ben-David).

✔ But in practice, SGD works well

Why?

- Not fully understood → still an open problem
- Many heuristics help:
    1. Momentum (Adam, RMSProp)
    2. Preconditioning
    3. Batch normalization
    4. Layer normalization

✔ Only well-understood case

Training is provably easy when:

- Width → infinity
- Depth = 1
  → Equivalent to kernel methods.

Many other convergence proofs essentially reduce the NN to this special case.

Lec – 32 Backpropagation

⭐ **Backpropagation — Explained Clearly and Simply**

Backpropagation is the algorithm used to compute **gradients** in neural networks so we can run **SGD**.

For linear and kernel models, gradients are easy to compute.
But for neural networks, the model is **deep** (many compositions), so the gradient is not straightforward.

Still, gradient computation is possible using an efficient strategy called **backpropagation**.

---

1. Why gradient computation is harder in neural nets?

A neural network function is typically:

$$f(w) = f_d(f_{d-1}(\dots f_2(f_1(w))))$$

This is a **deep composition** of functions.

- (f_1, f_2,…….,f_d) are layers.
- (w) are model parameters (weights in all layers).

Computing derivatives of long compositions requires **repeated chain rule**.

---

2. Forward Definitions

Define:

- (w_1 = w)
- (w_2 = f_1(w_1))
- (w_3 = f_2(w_2))
- …..
- (w_{d+1} = f_d(w_d) = f(w))

So:

✔ **Jacobian of the full function**

$$\frac{\partial f(w)}{\partial w} = \frac{\partial f_d(w_d)}{\partial w_d} \frac{\partial f_{d-1}(w_{d-1})}{\partial w_{d-1}} \cdots \frac{\partial f_1(w_1)}{\partial w_1}$$

This is the chain rule written in matrix form.

---

3. Two Ways to Compute the Gradient

You can compute this Jacobian by multiplying Jacobian blocks in two ways:

➡ **1. Forward Mode (Right-to-Left)**

Start from: $\frac{\partial f_1}{\partial w_1}$

and move forward through layers. -------But this becomes **very slow when output dimension is small**.

➡ **2. Backward Mode (Left-to-Right)**

Start from: $\frac{\partial f_d}{\partial w_d}$

and move backward. ---------- This is usually **much more efficient** in ML.

---

4. Why Backpropagation Is More Efficient

Key observation:

- The final function (f_d) is the **loss**.
- Loss output is **1-dimensional** (scalar).

So:

- Forward mode must propagate derivatives of **size equal to input dimension** (high).
- Backward mode propagates derivatives of **size = 1** (very small).

Thus:

🎯 **Backward computation always requires multiplying smaller matrices.**

→ faster
→ uses less memory

This is why backpropagation is the standard algorithm.

This backward application of the chain rule is called:

✔ **Backpropagation OR backprop.**

---

5. Backprop for General DAGs

The above chain-rule example was for a simple chain:

$$f_d \circ f_{d-1} \circ \cdots \circ f_1$$

But real neural networks can be general Directed Acyclic Graphs (DAGs):

- Skip connections
- Residual links
- Complex architectures

Section 13.3.4 of Murphy shows:

- how the same principle works on general DAGs
- gradients flow backward along all edges

This is why backprop is universal across all NN architectures.

---

6. Jacobians of Basic Units

We then computed the Jacobians for two key building blocks:

(a) Linear Unit

(b) ReLU Unit

(a) Linear Unit

From Murphy 13.3.3.3:

$$y = Wx$$

Jacobian wrt x:

$$\frac{\partial y}{\partial x} = W$$

Jacobian wrt parameter W:

$$\frac{\partial y}{\partial W} = x \text{ (with appropriate reshaping)}$$

(b) ReLU Unit

From Murphy 13.3.3.2:

$$a(x) = \max(0, x)$$

Derivative:

$$\frac{\partial a}{\partial x} = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

(0 or 1 applied elementwise)

Lec – 33 Over parameterized Models

📘 Chapter: Overparameterized Models — Simple & Complete Notes

Deep learning succeeds largely because of **modularity** (stacking simple layers). Modern LLMs have **billions of parameters** and show behaviors that classical theory did not predict. Many strange phenomena seen in deep learning later turned out to be general behaviors of **overparameterized models**, not something unique to neural networks.

Main observation:
When a model is **overparameterized** (number of parameters > number of samples), the usual **model error vs estimation error tradeoff breaks down**. Both can get better as model complexity increases — **double descent**.

---

1. What Are Overparameterized Models?

- These are models where the number of parameters **exceeds** the number of training samples.
- Example: Linear model with m > n.
- Such models can drive **training error to zero** (perfect interpolation).
- Classical theory (Chapter: Model vs Estimation Error) assumed **underparameterized regime** (m < n).
  → So the old tradeoff does **not** contradict the new observations.

---

2. ERM Solutions in the Overparameterized Regime

In under parameterized regime, the least squares solution is:

$$\hat{w}_{ls} = (DD^\top)^{-1}Dy$$

But when the model is overparameterized (n > m):

- There are **infinitely many** solutions to $D^\top w = y$.
- Any of them gives **zero empirical risk** (perfect fit).
- Most of these do **not generalize**.
  (This is "memorization" — Chapter ERM.)

To guarantee generalization we choose the solution with **minimum norm**.

---

3. Minimum-Norm Interpolant

The minimum-norm interpolant is:

$$\hat{w}_{ln} = D(D^\top D)^{-1} y$$

Why is this minimum norm?

Let $\hat{w}$ be any solution satisfying $D^\top \hat{w} = y$. Then:

$$\hat{w} - \hat{w}_{ln} \perp \hat{w}_{ln}$$

(This is shown using simple algebra and projection theorem.)

Hence:

$$\|\hat{w}\|^2 = \|\hat{w} - \hat{w}_{ln}\|^2 + \|\hat{w}_{ln}\|^2 \geq \|\hat{w}_{ln}\|^2$$

So $\hat{w}_{ln}$ is indeed the **minimum ℓ_2 norm** solution.

---

4. Expected Estimation Error of Min-Norm Interpolant

We study the estimation error:

$$\E[R[\hat{w}_{ln}]] - R[w^*]$$

Assumptions (same as earlier chapter, with one new simplification):

- Noise vector n is independent of X.
- All noises n_i are i.i.d.
- $\E[XX^\top] = I$ for simplicity.

We expand the estimation error:

$$\E[(\hat{w}_{ln} - w^*)^\top (\hat{w}_{ln} - w^*)]$$

After substituting $y = D^\top w^* + n$, the expression becomes:

Two main terms (bias + variance):

$$\E[n^\top (D^\top D)^{-1} n] + \E[(w^*)^\top (I - D(D^\top D)^{-1} D^\top) w^*]$$

The cross term is zero due to independence of noise.

---

5. Lower Bound on the Variance Term

We compute:

$$\E[n^\top (D^\top D)^{-1} n]$$

Using:

- $\E[nn^\top] = \E[N^2] I$
- Trace trick
- Jensen's inequality

We get:

$$\E[n^\top (D^\top D)^{-1} n] \geq \E[N^2] \frac{m}{n}$$

Notable fact:

👉 This is identical to underparameterized bound **but with roles of m & n swapped**.

---

6. Bias Term Calculation (Rotationally Invariant X)

Assume X is rotationally invariant (e.g., standard Gaussian). Then:

$$\E[(w^*)^\top D (D^\top D)^{-1} D^\top w^*] = \|w^*\|^2 \frac{m}{n}$$

Thus the bias term becomes:

$$\|w^*\|^2 - \|w^*\|^2 \frac{m}{n} = \|w^*\|^2 \left(\frac{n-m}{n} \right)$$

---

7. Combined Lower Bound

So estimation error is at least:

$$\|w^*\|^2 + (\E[N^2] - \|w^*\|^2) \frac{m}{n}$$

Interpretation:

- Overparameterized learning works **only if noise is low**:

$$\E[N^2] < \|w^*\|^2$$

If noise is high, error ≥ $\|w^*\|^2$ → terrible.

---

8. Exact Estimation Error (Special Case: Normal X)

When $D^\top D$ follows a Wishart distribution (Gaussian X), we know:

$$\E[(D^\top D)^{-1}] = \frac{m}{n-m-1}$$

So the exact estimation error is:

$$\|w^*\|^2 \left(\frac{n-m}{n}\right) + \E[N^2]\frac{m}{n-m-1} \qquad (\bigstar)$$

This formula is extremely important.

---

9. Minimizing the Exact Error (★)

Minimization w.r.t. `n` gives:

$$n = \frac{\|w^*\|}{\|w^*\| - \sqrt{\E[N^2]}}(m+1)$$

Only valid if:

$$\E[N^2] < \|w^*\|^2$$

The minimum estimation error becomes:

$$\|w^*\|^2 - \frac{m}{m+1}(\|w^*\| - \sqrt{\E[N^2]})^2$$

As $m \to \infty$, this tends to:

$$\|w^*\|^2 - (\|w^*\| - \sqrt{\E[N^2]})^2$$

It does **not** go to zero.

---

10. Interpretation: Why Overparameterization Works

Underparameterized case (m < n)

- Noise hurts you → must use regularization.
- Estimation error decreases as n → ∞.

Overparameterized case (m > n)

- Noise gets **absorbed** into parameters.
- We pick the solution that absorbs the **least** noise (min norm).
- Estimation error **plateaus**, not zero.
- But model error decreases with m → so generalization can still be excellent.

---

11. Key Differences Between the Two Regimes

Under parameterized	**Overparameterized**
Regularization optional	Regularization *necessary*
Hyperparameter controls regularization	No such clear hyperparameter
Estimation error → 0 as n → ∞	Estimation error plateaus
Best model at intermediate m (U-shape)	Best model also at large m (second U)

---

12. Double Descent (Two U-Shaped Curves)

Generalization error vs model size shows:

1. **Classical U-shape** (underparam.)
2. **Peak at interpolation threshold** (m = n)
3. **Second U-shape** (overparam.)
   → model error decreases as m→∞
   → estimation error plateaus

Diagrams shown in Fig. 1 & Fig. 2 (from the original notes).

---

13. Final Conclusions

- Overparameterization can outperform underparameterization in low-noise settings.
- Min-norm interpolants generalize surprisingly well.
- **Double descent** explains modern deep learning behavior.
- Key requirement:
  → either **explicit** or **implicit regularization** (e.g., SGD has implicit bias).
- Understanding both regimes together is an important open theoretical problem.

Lec – 34 Non-Parametric Models: k-Nearest Neighbour

Non-Parametric Models: (k)-Nearest Neighbour (Clean + Concise Version)

Models like linear regression and neural networks are **parametric models**.
They learn a set of parameters that *fully encode* the task. After training:

- Only the parameters are used for prediction
- Training data is not used during inference
- Information from training samples is stored **implicitly** in parameters

In contrast, models like kernel methods use:

$$\hat{f}(x) = \sum_{i=1}^{m} \alpha_i \kappa(x_i, x)\, y_i$$

These methods must **store the training data** and use it during inference.
They are called:

- **Non-parametric models**, or
- **Exemplar-based models**

Because they explicitly memorize training data:

1. They behave like overparameterized models: training loss is usually **zero** (except kernel methods with strong regularization).
2. As in overparameterized models, **regularization or assumptions** are needed for generalization.

---

1. Nearest Neighbour (NN) Classification

The nearest neighbour method is the simplest non-parametric classifier.

To define NN, we need a **distance metric**: $d : \mathcal{X} \times \mathcal{X} \to \mathbb{R}_+$

This distance must satisfy the metric properties (non-negativity, symmetry, triangle inequality).

1.1 Defining the 1-NN Model

Let the model store a finite set:

$$\mathcal{S} = \{(x'_i, y'_i)\}_{i=1}^{s} \subset \mathcal{X} \times \mathcal{Y}$$

Given a query input (x):

1. Find the nearest stored point

$$i^*(x) \in \arg\min_{i=1,\ldots,s} d(x, x'_i)$$

2. Predict using its label:

$$f_{\mathcal{S}}(x) = y'_{i^*(x)}$$

The **1-NN model** is formally:

$$^1\mathcal{N}_d = \left\{ f \;\middle|\; \exists (x_i', y_i') \in \mathcal{X} \times \mathcal{Y},\ f(x_i') = y_i',\ f(x) = y_{i^*(x)},\ i^*(x) = \arg\min_i d(x, x_i') \right\} \qquad (19.1)$$

During training:

S = D

Thus, the model literally **memorizes the training set** → non-parametric.

1.2 Why Does 1-NN Work? (Theorem 19.3)

Theorem 19.3 (Shalev-Shwartz & Ben-David) shows that 1-NN generalizes under the key assumption:

As (m -> infinity), the label posterior at a point

$$p^*(y|x)$$

is essentially the same as the label posterior at its nearest neighbour

$$p^*(y|x_{i^*(x)})$$

Meaning:

- With many samples, the nearest neighbour of (x) lies extremely close to (x).
- So the nearest neighbour's label is like a fresh sample from (p^*(y|x)).
- 1-NN simply **copies** this label.

But even as (m -> infty):

- 1-NN can have risk up to **twice** the Bayes optimal risk.
- Thus it has **non-zero model error**.

To fix this, we use more neighbours → (k)-NN.

---

2. (k)-Nearest Neighbour Classification

Again, store:

$$\mathcal{S} = \{(x_i', y_i')\}_{i=1}^s$$

Given a query (x):

1. Find the **(k) nearest neighbours**
2. Take the **majority label** among these (k) neighbours
3. Output that majority label

The model is denoted:

$$^k\mathcal{N}_d$$

Why majority vote?

- The labels of the (k) neighbours are samples from $p^*(y|x)$
- The majority label = **mode** of these samples
- Mode is the Bayes optimal decision rule for **0–1 loss**

2.1 Consistency of k-NN (Theorem 19.5)

Theorem 19.5 (ShDa14):

As

$$k \to \infty, \qquad m \to \infty, \qquad \frac{k}{m^{d+1}} \to 0$$

the (k)-NN classifier converges to the **Bayes optimal classifier** → **zero generalization error**.

This shows that (k)-NN removes the model-error issue of 1-NN.

---

3. (k)-NN Regression

Same structure as classification.

Given a query (x):

- Take the **average** of the labels of the (k) nearest neighbours.

Reason:

- The neighbours provide samples from (p*(y|x))
- The average = empirical **mean**
- Mean is Bayes optimal for the **squared loss**

Thus (k)-NN regression is a non-parametric analogue of local averaging.

---

4. Curse of Dimensionality

Theorems 19.3 and 19.5 show that nearest neighbour methods are severely affected by the **curse of dimensionality**.

In high-dimensional spaces:

- Points are rarely “close”
- Distances become uninformative
- Generalization error decreases extremely slowly:

$$O\left(\frac{1}{m^{n+1}}\right)$$

where

- (m) = number of samples
- (n) = dimension of input space

Also:

- NN search becomes computationally expensive in high dimensions.

Therefore:
Nearest neighbour methods are typically used only **after dimensionality reduction**.

Lec – 35 Non-Parametric Models: Kernel Density Estimation (KDE)

Non-Parametric Models: Kernel Density Estimation (KDE)

Nearest neighbour methods are examples of **non-parametric, non-probabilistic** models.
In this lecture, we discuss the **probabilistic** counterpart: **non-parametric likelihood estimation**.

---

1. Non-Parametric Likelihood Models

1.1 Discrete Case

If (Y) is discrete, the simplest non-parametric likelihood model is the **empirical distribution**:

$$\hat{p}(y) = \text{fraction of occurrences of } y \text{ in the training set}$$

Similarly, a **non-parametric conditional likelihood** can be constructed using the (k)-NN idea:

$$\hat{p}(y|x) = \text{fraction of the } k \text{ nearest neighbours of } x \text{ with label } y$$

These are natural and effective for discrete labels.

1.2 Continuous Case: Problem with Empirical Likelihood

If (Y) is continuous, the empirical distribution places **point-mass (Dirac delta) spikes** at observed samples:

- Assigns **zero probability** to any point not in the training set
- Useless for ML tasks (e.g., generalization, generative modelling)

To fix this, we **smooth** the spikes → KDE.

---

2. Kernel Density Estimation (KDE)

Instead of a Dirac impulse at each sample (y_i), place a **smooth kernel** (e.g., a Gaussian) centered at (y_i).

Thus, the KDE for $p(y)$ is:

$$\hat{p}(y) = \frac{1}{m}\sum_{i=1}^{m}\kappa(y - y_i)$$

Example: Gaussian kernel in n dimensions

$$\kappa(y) = \frac{e^{-\frac{1}{2}\|y\|^2}}{(2\pi)^{n/2}}$$

This gives a smooth probability density.

2.1 Properties of a Valid Density Kernel

A **density kernel** $\kappa(y)$ must satisfy:

1. $\kappa(y) \geq 0$ for all $y \in \mathcal{Y}$
2. $\int_{\mathcal{Y}} \kappa(y)\, dy = 1$
3. **Symmetry**: $\kappa(y) = \kappa(-y)$

Consequences:

- $\int_{\mathcal{Y}} y\kappa(y)\, dy = 0$
- $\int_{\mathcal{Y}} y\kappa(y - \mu)\, dy = \mu$

2.2 Bandwidth Parameter

Smoothness is controlled by the **bandwidth** parameter (h):

$$\kappa_h(y) = \frac{1}{h}\kappa\left(\frac{y}{h}\right)$$

Effects:

- (h → 0): kernel becomes sharp → KDE ≈ empirical distribution
- Larger (h): smoother density estimate

2.3 Consistency and Curse of Dimensionality

As (m -> infty), KDE converges to the true density

However:

- Density estimation is extremely hard in high dimensions
- KDE estimation error decays like:

$$O\left(\frac{1}{m^{n+2}}\right)$$

where
(m) = samples,
(n) = dimensionality.

This is the curse of dimensionality.
Importantly:

KDE is *not* inefficient — density estimation itself is fundamentally hard.
No estimator can be significantly better than KDE.

---

3. KDE-Based Generative Models

In supervised learning, to build a **generative model** using KDE, we need a kernel over the **joint space** X×Y

3.1 Constructing a Joint Kernel

Take kernels:

- κ_1 over $\mathcal{X}$
- κ_2 over $\mathcal{Y}$

Define:

$$\kappa(x, y) = \kappa_1(x)\,\kappa_2(y)$$

This satisfies all kernel properties, so it is a valid **joint density kernel**.

This satisfies all kernel properties, so it is a valid **joint density kernel**.

3.2 KDE for the Joint Distribution

$$\hat{p}(x, y) = \frac{1}{m}\sum_{i=1}^{m} \kappa_1(x - x_i)\,\kappa_2(y - y_i)$$

Marginals

$$\hat{p}(x) = \frac{1}{m}\sum_{i=1}^{m} \kappa_1(x - x_i)$$

$$\hat{p}(y) = \frac{1}{m}\sum_{i=1}^{m} \kappa_2(y - y_i)$$

Thus, **marginals are simply KDEs with their respective kernels.**

Notice:

$$\hat{p}(x, y) \neq \hat{p}(x)\hat{p}(y)$$

→ the KDE joint model is **non-trivial** and captures correlations.

---

4. Conditional KDE and Nadaraya–Watson Estimator

Conditional likelihood:

$$\hat{p}(y|x) = \frac{\hat{p}(x,y)}{\hat{p}(x)} = \frac{\frac{1}{m}\sum_{i=1}^{m}\kappa_1(x-x_i)\kappa_2(y-y_i)}{\frac{1}{m}\sum_{i=1}^{m}\kappa_1(x-x_i)}$$

Bayes Optimal Predictor (Posterior Mean)

$$\int_y y\hat{p}(y|x)\,dy = \frac{\sum_{i=1}^{m}\kappa_1(x-x_i)\int_y y\kappa_2(y-y_i)\,dy}{\frac{1}{m}\sum_{i=1}^{m}\kappa_1(x-x_i)}$$

Using the kernel property:

$$\int y\kappa_2(y-y_i)\,dy = y_i$$

Hence:

$$\int_y y\,\hat{p}(y|x)\,dy = \sum_{i=1}^{m}\bar{\kappa}_{\mathcal{D}}(x-x_i)\,y_i$$

where

$$\bar{\kappa}_{\mathcal{D}}(x-x_i) = \frac{\kappa(x-x_i)}{\sum_{j=1}^{m}\kappa(x-x_j)}$$

This gives the **Nadaraya–Watson estimator** — a KDE-based regressor that:

- Interpolates training labels
- Weighs each label by **similarity** between (x) and (x_i)

Important:
KDE regression only interpolates.
Unlike kernel regression (Eq. 16.x), it **cannot form arbitrary linear combinations**.

---

5. Connection to Attention

Attention mechanisms in modern LLMs can be interpreted as a form of **KDE regression**:

- Query–key similarity → kernel weights
- Value vectors → labels
- Output = weighted sum like Nadaraya–Watson

Lec – 36 Generative AI

Generative AI — Clean & Concise Notes (No Loss of Content)

Generative AI (GenAI) is concerned with producing new samples that look like they were drawn from an unknown distribution.

Definition

Given m samples from an unknown likelihood p, generate more samples from p

This is related to likelihood estimation but **not identical**:

- The problem definition **does not require** estimating the likelihood.
- But **if** we estimate the likelihood, then GenAI reduces to:
  → writing a sampler that draws from the estimated (p).

Methods that **avoid estimating the likelihood** are called:

- **Implicit generative methods**
  (direct generation; covered in advanced course CS6090)

Methods that **estimate the likelihood and then sample** are:

- **Explicit generative methods**
  (the approach followed in this course)

---

1. Energy-Based Models (EBMs)

Recall exponential family models:

$$p_w(x) = \frac{e^{w^\top \phi(x)}}{Z(w)}.$$

These are normalized linear models.

A deep-learning analogue is obtained by replacing $w^\top \phi(x)$ with a neural-network energy function $\mathcal{E}_\theta(x)$:

$$p_\theta(x) = \frac{e^{-\mathcal{E}_\theta(x)}}{Z(\theta)}.$$

- Lower energy → higher likelihood
- $\mathcal{E}_\theta$ = neural network
- These are called **Energy-Based Models (EBMs)**
- They can be viewed as reparameterized exponential-family models

SGD for Maximum Likelihood in EBMs

SGD for Maximum Likelihood in EBMs

The update rule (analogous to earlier SGD derivations) is:

$$\theta^{(k+1)} = \theta^{(k)} - s_k \left(\nabla_\theta \mathcal{E}_\theta(x_{i_k}) - \mathbb{E}_{X \sim p_{\theta^{(k)}}} [\nabla_\theta \mathcal{E}_\theta(X)] \right)_{\theta=\theta^{(k)}} . \text{labeleqn :}$$

→ To compute the second expectation term, we **must sample** from p_θ.
→ We also need a sampler that works even when $Z(\theta)$ is unknown.

(See section 24.1 and eqns (24.7)–(24.16), *pml2Book*.)

---

2. Langevin Sampling

A quantity that **does not require** the normalizing constant is the mode:

$$\arg\max_x p_\theta(x) = \arg\min_x \mathcal{E}_\theta(x).$$

SGD on the energy directly finds the mode:

$$X^{(k+1)} = X^{(k)} - s_k \nabla_x \mathcal{E}_\theta(x)\big|_{x=X^{(k)}} + s_k N_k, \quad N_k \sim \mathcal{N}(0, I).$$

where the gradient is w.r.t. **x**, not (\theta).

The quantity

$$\nabla_x \log p(x)$$

is known as the **Stein score**.

(Meanwhile, $\nabla_\theta \log p_\theta(x)$ is the Fisher score.)

However our goal is **not** to reach the mode → but to obtain **samples** from (p_\theta).

The Correct Sampling Dynamics

Using SDE theory, the correct sampling update becomes:

$$X^{(k+1)} = X^{(k)} - s\nabla_x \mathcal{E}_\theta(x)\big|_{x=X^{(k)}} + \sqrt{2s}\, N_k, \quad N_k \sim \mathcal{N}(0, I).$$

This is **Langevin sampling**.

- Straightforward to implement
- Works even when (Z(Theta)) is unknown
- Figure 25.5 in *pml2Book* shows an illustration
- Modern diffusion models (e.g., DDPM) are based on similar ideas

Sampling Procedure

1. **Initialize** $X^{(0)}$ randomly
   (for images: pure noise image)
2. For each iteration:
   - sample $N_k \sim \mathcal{N}(0, I)$
   - update using Eq. \eqref{eqn:langsamp}
   - compute Stein score via backprop through $\mathcal{E}_\theta$
3. After many iterations:
   $X^{(k)}$ becomes a sample from p_θ → a denoised/generated image

---

3. Convergence intuition via Gaussian example

Consider Gaussian target density:

$$p_\theta(x) \propto \exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right).$$

If step-size s is small, then:

Mean evolution

$$\E[X^{(k)}] = \mu + \underbrace{\left(I - s\Sigma^{-1}\right)^k \left(\E[X^{(0)}] - \mu\right)}_{\text{decays exponentially to } 0}$$

Covariance evolution

$$\operatorname{cov}(X^{(k)}) \approx \Sigma + \underbrace{\frac{s}{2}I}_{\text{bias linear in } s} + \underbrace{\left(I - s\Sigma^{-1}\right)^{2k}\left(\operatorname{cov}(X^{(0)}) - \Sigma - \frac{s}{2}I\right)}_{\text{exponentially decaying error}}$$

Interpretation:

- Mean → converges exponentially to μ, provided $s < \lambda_{\min}(\Sigma)$.
- Covariance → converges exponentially to $\Sigma + \frac{s}{2}I$.
  As $s \to 0$, this bias disappears.

This limit with (s \to 0) corresponds to **Langevin diffusion** (continuous-time version).

---

4. Conditional Sampling (Generative AI with Conditions)

Given i.i.d. samples

$$\mathcal{D} = \{(x_1, y_1), \ldots, (x_m, y_m)\} \sim p(x, y),$$

we may want to sample from the **conditional distribution** $p(y|x)$.
Example: image generation conditioned on text caption.

We can apply the same EBM + Langevin approach.

Procedure

1. Define conditional energy-based model:

$$p_\theta(y|x) \propto e^{-\mathcal{E}_\theta(x,y)}.$$

2. Train via MLE to obtain $\hat{\theta}$.
   (Training again requires Langevin sampling steps.)
3. For a given conditioning x:
   - compute **Stein score**

$$\nabla_y \log p_{\hat{\theta}}(y|x)$$

   - run Langevin sampling in y-space

After enough iterations:
→ the final y is approximately a sample from $p(y|x)$.

Lec – 37 Online Learning

Online Learning — Clean & Concise Version (All Content Preserved)

We want to model a form of learning similar to how humans (e.g., a baby) learn:

- The mother shows an image.
- The baby predicts what it is.
- Only *after* predicting, the correct answer is revealed.
- This repeats with new inputs.

This captures the core idea of **online learning**:
The learner must make a prediction *before* the supervision (label) is revealed.

Because supervision arrives *after* the prediction:

- The learner cannot memorize future labels (they are unknown).
- The relevant performance measure is the number of mistakes or average loss **during the interaction**, not after seeing all data as in batch learning.

---

1. Online Supervised Learning Setup

We consider:

- Model class / inductive bias: (F), parametrized by (w)
- Loss function (l)
- Online algorithm (A)

At each round (t):

1. Input x_t arrives
2. The learner predicts using $f_{w_t}(x_t)$
3. True label y_t is revealed
4. The learner updates to w_{t+1}

The **online risk** after (T) rounds is

$$\frac{1}{T}\sum_{t=1}^{T} l\left(f_{w_t}(x_t), y_t\right).$$

Lower online risk means a better online algorithm.

---

2. How to Measure Success?

In supervised learning the goal is to find the Bayes **optimal f^*.**

But in online learning:

- The algorithm outputs a *sequence* of models $(w_1, \dots, w_T, \dots)$
- We cannot require $f_{w_T} \to f^*$ because different Bayes optima may exist

- We also cannot demand low risk only after a very large T_0(e.g., LLMs that learn only after massive training)

Thus, we compare the online algorithm's performance to the Bayes risk using:

$$\frac{1}{T}\sum_{t=1}^{T} l(f_{w_t}(x_t), y_t) - R[f^*].$$

This difference can be **decomposed** as:

$$\begin{aligned}
&\frac{1}{T}\sum_{t=1}^{T} l(f_{w_t}(x_t), y_t) - R[f^*] \\
&= \frac{1}{T}\left(\underbrace{\sum_{t=1}^{T} l(f_{w_t}(x_t), y_t) - \min_{w}\sum_{t=1}^{T} l(f_w(x_t), y_t)}_{\text{Regret}}\right) \\
&+ \underbrace{\min_{w}\frac{1}{T}\sum_{t=1}^{T} l(f_w(x_t), y_t) - \min_{w}\mathbb{E}[l(f_w(X), Y)]}_{\text{Estimation Error}} \\
&+ \underbrace{\min_{w}\mathbb{E}[l(f_w(X), Y)] - \min_{f}\mathbb{E}[l(f(X), Y)]}_{\text{Model Error}} \backslash \text{label} eqn : onldeco
\end{aligned} \tag{1}$$

The **new quantity** here is the **regret**:

- Regret may be negative, but typically is positive
- It captures the disadvantage of predicting *without* seeing the label
- Controlling regret becomes **the main goal** of online learning

---

3. Relationship to Batch Learning

Both batch and online learning are about "optimizing under uncertainty":

- In batch learning, uncertainty is **stochastic** (labels sampled i.i.d.)
- In online learning, uncertainty is **availability-based** (future labels unknown)

Important distinction:

- i.i.d. assumptions **help with estimation error**, but
- i.i.d. assumptions **do not help with regret**
  (because both comparator and algorithm face the same observed points)

Thus, online learning is treated as a **separate, self-contained problem**:
→ controlling regret.

An algorithm is good if its regret grows **sublinearly**, e.g.:

- $O(\sqrt{T})$
- $O(\log\ T)$

For 0–1 loss, sublinear regret means the number of mistakes is (o(T)), which is intuitive.

When regret is small, the online risk approaches Bayes risk:

$$\frac{1}{T}\sum_{t=1}^{T} l(f_{w_t}(x_t), y_t) - R[f^*] \leq \epsilon$$

for large enough (T).

---

4. Inductive Bias ⇒ Online Learning

We now illustrate why **inductive bias** (model assumptions) is essential.

Example: Boolean functions

Let:

- Inputs: (n)-bit vectors
- Function class: all Boolean functions on (n)-bit inputs
  → total of 2^{2^n} functions

Online goal: identify the unknown target (f^*).

At each round:

1. An input (x_t) appears
2. Learner predicts (f^*(x_t))
3. Feedback is revealed
4. Repeat

Case 1: No inductive bias (F is maximally large)

Worst case:

- The learner may be wrong on **every round**
- After each mistake, they eliminate inconsistent functions
- They need **(2^n)** rounds before seeing repeated inputs
- This is just **memorization**, not learning
- Regret cannot be controlled

Conclusion:
If nothing is known about the future, regret control is impossible.

Case 2: Restrict $|\mathcal{F}| < 2^{2^n}$

A naïve (non-murkha) learner:

- Eliminates functions inconsistent with past outputs
- Still may suffer $|\mathcal{F}|$ mistakes in the worst case
- Again, purely memorizing; not efficient

Case 3: A "Kuśāgrabuddhi" (efficient) learner

The smart learner:

- Must be consistent with past observations
- **Also plans for uncertainty about the future**

At round (t):

- Consistent functions split into two sets:
  - Functions predicting 0 at (x_t)
  - Functions predicting 1 at (x_t)

Strategy:

1. Look at which set is **larger**
2. Predict the label corresponding to the **larger set**
3. Worst case: your prediction is wrong → eliminate the **larger** set → maximum possible reduction
4. Best case: you predict correctly → eliminate the smaller set

This strategy yields regret:

$$\text{Regret}(t) = \begin{cases} t, & t \leq \log(|\mathcal{F}|), \\ \log(|\mathcal{F}|), & \text{otherwise.} \end{cases}$$

This is **much better** than $|\mathcal{F}|$.
This is an *efficient learner*.

---

5. Key Takeaways

1. **Statistical (i.i.d.) assumptions help estimation error, but not regret.**
   Regret control is fully independent of data being i.i.d.
2. **Consistency (ERM) is necessary but not sufficient in online learning.**
   Past goodness does not imply future goodness.
3. **Must plan for worst-case future uncertainty.**
   Unlike batch learning, where stochastic assumptions rescue us.
4. **Inductive bias is essential.**
   Some function classes make regret control impossible; others (e.g., convex Lipschitz functions — next lecture) allow efficient regret bounds.

Lec – 38 Online Convex Optimization

---

1. Definition of Online Optimization

At each time step $t = 1, 2, \ldots, T$:

1. The learner picks a decision

$$w_t \in \mathcal{W}.$$

2. After the decision is made, a **loss function**

$$f_t : \mathcal{W} \to \mathbb{R}$$

is revealed (or partial information about it).

3. The learner suffers loss $f_t(w_t)$.

The goal is to minimize **regret**:

$$\sum_{t=1}^{T} f_t(w_t) - \min_{w \in \mathcal{W}} \sum_{t=1}^{T} f_t(w). \tag{1}$$

A good online algorithm achieves **sublinear regret**, e.g., $o(T)$.

Connection to Online Supervised Learning

If we interpret

$$f_t(w_t) \equiv l(f_{w_t}(x_t), y_t)$$

then OCO exactly recovers the online learning setup from the previous lecture.

---

2. Levels of Information

Depending on what feedback is available after choosing (w_t), we get different subfields:

1. Bandit Optimization (Zero-order)

- Only the value ft(wt) is observed
- No gradient information

2. First-order Online Optimization

- Only the gradient (or subgradient)
  ∇ft(wt) is revealed

3. Full-information Online Optimization

- Entire function (f_t) is known afterward

Relation to Learning Types

- Batch supervised learning → optimization error
- Online supervised learning → regret term

---

3. Online Stochastic Optimization

If (f_t) are i.i.d. samples from some underlying likelihood (like in supervised learning), then traditional regret does not make sense. Instead, we consider:

$$\sum_{t=1}^{T} l(f_{w_t}(x_t), y_t) - T \min_{w} \mathbb{E}[l(f_w(X), Y)]. \qquad (2)$$

This is called **regret** in online stochastic optimization.

In summary:

- **Batch supervised learning = stochastic optimization**
- **Online supervised learning = full-information online stochastic optimization**

---

4. When Is Online Optimization Possible?

Online optimization is tractable only for certain function classes.

A key tractable class:

- **Convex** functions
- **Lipschitz** functions

This follows from the same machinery used to analyze **SGD**.

Given first-order information (gradients), OCO becomes feasible.

---

5. Online Gradient Descent (OGD)

Consider convex, Lipschitz losses $f_t \in \mathcal{F}$.

We run:

$$w_{t+1} = w_t - s\nabla f_t(w_t).$$

This is exactly the stochastic gradient descent update when attempting to minimize:

$$\min_{w} \sum_{t=1}^{T} f_t(w)$$

in a batch setting (where sampling is uniform over the set $\{f_1, \ldots, f_T\}$).

The same update **can be used online**, yielding the **Online Gradient Descent (OGD)** algorithm.

---

6. Regret of OGD (Full Proof Preserved)

We restate Theorem 21.15 from Shalev-Shwartz & Ben-David.

Let

$$w^* = \arg\min_w \sum_{t=1}^{T} f_t(w).$$

We analyze:

$$\|w_{t+1} - w^*\|^2 - \|w_t - w^*\|^2.$$

Using OGD update:

$$w_{t+1} = w_t - s\nabla f_t(w_t),$$

expand:

$$\begin{aligned}
&\|w_{t+1} - w^*\|^2 - \|w_t - w^*\|^2 \\
&\quad = \|(w_t - w^*) - s\nabla f_t(w_t)\|^2 - \|w_t - w^*\|^2 \\
&\quad = s^2\|\nabla f_t(w_t)\|^2 - 2s\nabla f_t(w_t)^\top (w_t - w^*) \\
&\quad \le s^2 L^2 - 2s\nabla f_t(w_t)^\top (w_t - w^*) \\
&\quad \le s^2 L^2 - 2s(f_t(w_t) - f_t(w^*)).
\end{aligned}$$

Where:

- First inequality uses **L-Lipschitzness**
- Second uses **convexity**

Summing over $t = 1, \ldots, T$:

$$\sum_{t=1}^{T}(f_t(w_t) - f_t(w^*)) \le \frac{1}{2}\left(sTL^2 + \frac{\delta_1^2 - \delta_{T+1}^2}{s}\right)$$

where $\delta_t = \|w_t - w^*\|$.

Ignoring the negative term:

$$\text{Regret(OGD)} \le \frac{1}{2}\left(sTL^2 + \frac{\delta_1^2}{s}\right).$$

Choose step-size:

$$s^* = \frac{\delta_1}{L\sqrt{T}}.$$

Then:

$$\text{Regret} \le \delta_1 L\sqrt{T}.$$

This is **sublinear regret** → OGD is an **efficient online algorithm**.

---

7. Online-to-Batch Conversion

If all $f_t = f$, then by Jensen's inequality:

$$f\left(\frac{1}{T}\sum_{t=1}^{T} w_t\right) - f(w^*) \le \frac{1}{T}\sum_{t=1}^{T}(f(w_t) - f(w^*)) \le \frac{\delta_1 L}{\sqrt{T}}.$$

This gives convergence rates for:

- **SGD on true risk**
- **SGD on empirical risk (ERM + SGD)**

→ This reproduces the first row of Table

Thus:

If online learning is possible for a model class, then batch learning is also possible.

But the converse need not hold.
(Example: 1D linear classifiers.)

Lecture 39 Averaging (Ensembling)

Averaging (Ensembling) — Clean & Concise Notes (All Content Preserved)

A fundamental idea in statistics is that **averaging reduces variance**:
by the Law of Large Numbers, the sample mean becomes more accurate as we collect more samples.

We now apply this idea to **ML model estimates**.

Suppose:

- We train the same model (k) times
- Using (k) independent datasets from the same underlying distribution
- We obtain (k) estimators:

$$\hat{f}_1, \hat{f}_2, \ldots, \hat{f}_k.$$

We then form the **ensemble / averaged estimator**:

$$\hat{f} = \frac{1}{k}\sum_{i=1}^{k} \hat{f}_i.$$

Question: Is this averaged estimator better?

We analyze this in the context of **squared loss**, where Bayes optimal is

$$f^*(x) = \mathbb{E}[Y|X = x].$$

---

1. Generalization Error for Squared Loss

Generalization error:

$$R[\hat{f}] - R[f^*] = \mathbb{E}[(\hat{f}(X) - Y)^2] - \mathbb{E}[(f^*(X) - Y)^2].$$

Expand:

$$\begin{aligned} &= \mathbb{E}[(\hat{f}(X) - f^*(X))^2] \\ &\quad + 2\mathbb{E}[(\hat{f}(X) - f^*(X))(f^*(X) - Y)]. \end{aligned}$$

Since

$$f^*(X) = \mathbb{E}[Y|X],$$

the cross term vanishes:

$$2\mathbb{E}[(\hat{f}(X) - f^*(X))(f^*(X) - Y)] = 0.$$

Thus:

$$R[\hat{f}] - R[f^*] = \mathbb{E}[(\hat{f}(X) - f^*(X))^2].$$

So generalization error = **estimation error**.

---

2. Bias–Variance Decomposition

Since $\hat{f}$ is random (depends on dataset $\mathcal{D}$), let:

$$\mathbb{E}_{\mathcal{D}}[\hat{f}(x)]$$

denote the expectation over datasets.

Then:

$$\begin{aligned} \mathbb{E}_{\mathcal{D}}\mathbb{E}[(\hat{f}(X) - f^*(X))^2] = \mathbb{E}\left[\underbrace{\mathrm{var}_{\mathcal{D}}(\hat{f}(X))}_{variance}\right] \\ + \mathbb{E}\left[\underbrace{(\mathbb{E}_{\mathcal{D}}[\hat{f}(X)] - f^*(X))^2}_{bias}\right]. \end{aligned}$$

This is the classical **bias–variance decomposition**.

- Large models → low bias but high variance
- Small models → high bias but low variance
- ML's *model/estimation error tradeoff* is often described in these terms

---

3. Bias and Variance of the Averaged Estimator

Because $\hat{f}_i$ are iid:

Bias

$$\mathbb{E}_{\mathcal{D}}[\hat{f}(X)] = \frac{1}{k}\sum_{i=1}^{k}\mathbb{E}_{\mathcal{D}}[\hat{f}_i(X)] = \mathbb{E}_{\mathcal{D}}[\hat{f}_1(X)].$$

So **averaging does NOT change bias**.

Variance

Since the $\hat{f}_i$ are independent:

$$\mathrm{var}(\hat{f}) = \frac{1}{k^2}\sum_{i=1}^{k}\mathrm{var}(\hat{f}_i) = \frac{1}{k}\mathrm{var}(\hat{f}_1).$$

Thus variance **shrinks by factor** $1/k$.

So ensembling improves generalization **by reducing variance**.

---

4. But in Practice We Have Only One Dataset

Given only *one* dataset of size (m), we have two options:

1. Use full dataset → train $\hat{f}$
2. Split dataset into k equal parts → train
   $\hat{f}_1, \ldots, \hat{f}_k$
   and average them

Which is better depends on whether we are in the **under parameterized** or **overparameterized** regime.

We now analyse both regimes *fully and rigorously*, preserving all derivations.

---

5. Under parameterized Regime

Estimator:

$$\hat{f}^{ls}(x) = y^\top D^\top (DD^\top)^{-1}\phi(x) = w^{*\top}\phi(x) + n^\top D^\top (DD^\top)^{-1}\phi(x).$$

Assumptions from the earlier chapter apply:

- $\mathbb{E}[N] = 0$
- N independent of X

Thus:

$$\mathbb{E}_D[\hat{f}^{ls}(x)] = f^*(x).$$

So **least squares is unbiased**.

From earlier:

- Expected variance lower bound: $\frac{n}{m}$
- Expected variance upper bound: $\sqrt{\frac{n}{m}}$

We examine both "easy" (lower bound achieved) and "hard" (upper bound) cases.

5.1 Easy Case (Lower Bound)

Variance of $\hat{f}_i^{ls}$ when trained on m/k samples:

$$\frac{n}{m/k} = \frac{kn}{m}.$$

Variance of averaged estimator:

$$\frac{1}{k} \cdot \frac{kn}{m} = \frac{n}{m}.$$

→ **Same variance as using the full dataset.**

Averaging gives **no benefit**.

Reason: Least squares is already optimal in this regime (attains the lower bound).

5.2 Hard Case (Upper Bound)

Variance upper bound of each estimator:

$$\sqrt{\frac{n}{m/k}} = \sqrt{\frac{nk}{m}}.$$

Variance of averaged estimator:

$$\frac{1}{k}\sqrt{\frac{nk}{m}} = \sqrt{\frac{n}{mk}}.$$

To maximize averaging benefit, choose largest possible k:

$$\frac{m}{k} \geq n \quad \Rightarrow \quad k \leq \frac{m}{n}.$$

Plugging $k = \frac{m}{n}$:

$$\sqrt{\frac{n}{m(m/n)}} = \frac{n}{m}.$$

Thus averaging can convert the hardest problems into the easiest ones.

→ **Huge improvement in generalization.**

5.3 Special Gaussian Case (From Bach)

For Gaussian inputs:

$$\mathrm{var}(\hat{f}^{ls}) = \mathbb{E}[N^2]\frac{n}{m-n-1}.$$

For each split (size m/k):

$$\mathrm{var}(\hat{f}_i^{ls}) = \mathbb{E}[N^2]\frac{n}{m/k-n-1}.$$

Variance of ensemble:

$$\mathrm{var}(\tilde{f}_k^{ls}) = \frac{1}{k}\mathbb{E}[N^2]\frac{n}{m/k-n-1} = \mathbb{E}[N^2]\frac{n}{m-kn-k}.$$

This is **always larger** than the full-dataset variance for
$1 < k < m/(n+1)$.

→ Averaging **always worsens generalization** in this special case.

---

6. Overparameterized Regime

Estimator:

$$\hat{f}^{ln}(x) = y^\top (D^\top D)^{-1} D\phi(x)$$

(minimum-norm interpolant)

General facts (from earlier):

- Estimator is **biased**
- Expression decomposes estimation error into bias + variance

Expected generalization error:

Expected generalization error:

$$\underbrace{\|w^*\|^2 \left(\frac{n-m}{n}\right)}_{\text{bias}} + \underbrace{\mathbb{E}[N^2]\frac{m}{n-m-1}}_{\text{variance}}.$$

Now consider splitting:

Bias after averaging

$$\|w^*\|^2 \left(\frac{n-m/k}{n}\right)$$

→ **Bias increases** as k grows.

Variance after averaging

Variance of each model:

$$\mathbb{E}[N^2]\frac{m/k}{n-m/k-1}$$

Variance of averaged ensemble:

$$\frac{1}{k}\mathbb{E}[N^2]\frac{m/k}{n-m/k-1}.$$

→ **Variance decreases** when averaging.

So, in overparameterization:
Increasing (k)

- reduces variance
- increases bias
  → a **true bias-variance tradeoff**

The optimal (k) minimizes:

$$\|w^*\|^2 \frac{n-m/k}{n} + \mathbb{E}[N^2]\frac{m}{k(kn-m-k)}.$$

The quadratic form in (k) (with coefficients (a,b,c)) follows exactly as in the original text.

---

7. Summary of When Averaging Helps

- For "easy" underparameterized problems: **no benefit**
- For some special cases (Gaussian): **averaging worsens generalization**
- For "hard" underparameterized problems: **massive improvement**
- For overparameterized models:
  → **tradeoff; best (k) must be chosen carefully**

Because the analysis is complex, (k) is often selected via **cross-validation**.

---

8. Practical and Computational Notes

- Larger (k) improves computation:

    - Each model trains on (m/k) samples
    - All models can be trained in parallel
- But too large (k) may worsen generalization
- These ideas underpin **federated learning**
- Bagging is a further generalization
  (see Murphy, Bach for more details)

Lecture 40 Boosting

Boosting — Clean & Concise Notes (All Content Preserved)

In batch supervised learning, our main challenge has always been **controlling generalization error**, which decomposes into:

- **bias** (model error)
- **variance** (estimation error)

Previously, our approach was *liberal*:
choose **large, universal models** (kernels, deep nets), then use **regularization / ensembling** to control variance.

Boosting takes the opposite, **conservative** ("ahimsic") approach:

Start with an extremely simple model class, and then incrementally make it more expressive by adding small corrections.

This is "boosting."

---

1. Basic Idea of Boosting

We begin with a **minimalist base model** class (F):

- e.g., ReLU-activated linear units
- or even simpler: **decision stumps**

  $$f_{i,a}(x) = \text{sign}(x_i - a)$$

  parameters: (i,a)

Individually, these weak models may perform poorly.
But their **linear span**:

$$\text{span}(\mathcal{F})$$

is very expressive (e.g., single-layer ReLU networks can approximate any function).

So boosting builds models **sequentially**:

$$\begin{aligned} f^1(x) &= \beta_1 f_{w_1}(x) \\ f^2(x) &= \beta_1 f_{w_1}(x) + \beta_2 f_{w_2}(x) \\ &\cdots \\ f^T(x) &= \sum_{t=1}^{T} \beta_t f_{w_t}(x) \end{aligned}$$

Key points:

- (T) (the width) is **not** fixed in advance.
- At stage (t), only β_t, w_t are optimized.
- Previously learned components are **kept fixed**.
- All **training samples** are used at each stage (unlike online learning).

The stage-wise optimization problem is:

$$\min_{\beta_t, w_t} \mathbb{E}\left[l(f^{t-1}(X) + \beta_t f_{w_t}(X), Y) \right] \qquad (1)$$

This greedy, sequential procedure is the core of boosting.

We now study two specific losses:

- **square loss → Matching Pursuit**
- **exponential loss → AdaBoost**

---

2. Square Loss (Matching Pursuit)

(Section 10.3.3 in Bach)

Using empirical risk, (1) becomes:

$$\min_{\beta_t, w_t} \sum_{i=1}^{m} \left(f^{t-1}(x_i) + \beta_t f_{w_t}(x_i) - y_i \right)^2$$

Expand the quadratic:

$$\beta_t^2 \sum_{i=1}^{m} f_{w_t}(x_i)^2 - 2\beta_t \sum_{i=1}^{m} f_{w_t}(x_i)(y_i - f^{t-1}(x_i))$$

Optimal β_t

$$\beta_t = \frac{\sum_{i=1}^{m} f_{w_t}(x_i)\left(y_i - f^{t-1}(x_i)\right)}{\sum_{i=1}^{m} f_{w_t}(x_i)^2}$$

This is exactly the **projection coefficient** of the residual

$$\delta_y^{t-1} = y - f^{t-1}$$

onto the vector $[f_{w_t}(x_1), \ldots, f_{w_t}(x_m)]^\top$.

Optimizing (w_t)

Substituting optimal β_t:

$$\arg\max_{w_t} \frac{\sum_{i=1}^m f_{w_t}(x_i)\,(y_i - f^{t-1}(x_i))}{\sqrt{\sum_{i=1}^m f_{w_t}(x_i)^2}}$$

Define normalized functions:

$$\bar{f}(x) = \frac{f(x)}{\sqrt{\sum_{i=1}^m f(x_i)^2}}, \quad \bar{\mathcal{F}}_{\mathcal{D}} = \{\, \bar{f}(\cdot) \text{ on training set} \,\}$$

Then the stage-wise problem is equivalent to:

$$\arg\min_{u_t \in \bar{\mathcal{F}}_{\mathcal{D}}} \|u_t - \delta_y^{t-1}\|^2$$

i.e., **project the residual onto** the normalized function dictionary.

Exponential Convergence

Residual update:

$$\|\delta_y^t\| = \|(I - P_{\bar{\mathcal{F}}_{\mathcal{D}}})\,\delta_y^{t-1}\| \le \|I - P_{\bar{\mathcal{F}}_{\mathcal{D}}}\|^t \|y\|$$

Thus residuals shrink **geometrically** → fast convergence.

If $\bar{\mathcal{F}}_{\mathcal{D}}$ is closed/convex, projection is well-defined.
This is the classical **Matching Pursuit / Dictionary Learning** viewpoint.

---

3. Exponential Loss (AdaBoost)

(Section 10.3.4 in Bach)

Loss:

$$l(y, f(x)) = e^{-yf(x)}.$$

Plug into (1) with empirical expectation:

$$\hat{R}[f^t] = \min_{\beta_t, w_t} \sum_{i=1}^m e^{-y_i(f^{t-1}(x_i) + \beta_t f_{w_t}(x_i))}$$

Factor:

$$\hat{R}[f^{t-1}] \cdot \min_{\beta_t, w_t} \sum_{i=1}^m \pi_i e^{-\beta_t y_i f_{w_t}(x_i)}$$

where the weights:

$$\boxed{\pi_i = \frac{e^{-y_i f^{t-1}(x_i)}}{\hat{R}[f^{t-1}]}}$$

Optimize over (βt)

Optimize over β_t

Partition data into:

- errors: $y_i \neq f_{w_t}(x_i)$
- correct: $y_i = f_{w_t}(x_i)$

Define weighted error:

$$\varepsilon_t = \sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i$$

Then:

$$\min_{\beta_t} \left[e^{\beta_t} \varepsilon_t + e^{-\beta_t}(1 - \varepsilon_t) \right]$$

Optimal:

$$\boxed{\beta_t = \frac{1}{2} \log \frac{1 - \varepsilon_t}{\varepsilon_t}}$$

Optimal objective:

$$2\sqrt{\varepsilon_t(1 - \varepsilon_t)}$$

Thus:

$$\hat{R}[f^t] = 2\hat{R}[f^{t-1}] \min_{w_t} \sqrt{\varepsilon_t(1 - \varepsilon_t)}$$

---

4. When Does Boosting Improve the Model?

Boosting improves risk whenever:

$$\varepsilon_t < \frac{1}{2}.$$

If $\varepsilon_t > \frac{1}{2}$, simply flip the classifier:

$$f_{w_t} \mapsto -f_{w_t}$$

and since $\mathcal{F}$ is assumed symmetric ($f \in \mathcal{F} \Rightarrow -f \in \mathcal{F}$), this is allowed.

Thus the **only failure case** is:

$$\varepsilon_t = \frac{1}{2}.$$

---

5. Weak Learner Condition → Exponential Convergence

If we can guarantee:

$$\varepsilon_t \leq \frac{1}{2} - \epsilon \quad (\epsilon > 0),$$

then:

$$\hat{R}[f^t] \leq (1 - 2\epsilon)\hat{R}[f^{t-1}] \leq \cdots \leq (1 - 2\epsilon)^t \hat{R}[f^0].$$

→ Exponential decay of empirical risk.

This is the classical AdaBoost guarantee.

---

6. Extension to General Smooth Losses

Section 10.3.5 in Bach gives the generalization using smooth loss functions
(see equations (10.14), (10.15)).
The same stage-wise principle and convergence structure applies.
